

Musterlösung — Übungsklausur 2

Deep Learning 1 · TU Berlin

1 Multiple Choice

1. Translation Equivariance?

- ✓ (a) $f(T(x)) = T(f(x))$ — Translation am Eingang → gleiche Translation am Ausgang
- (b) Ausgang ändert sich nicht (= Invarianz)
- (c) Rotationsinvarianz
- (d) Pooling macht equivariant

2. Korrekte Aussage zu LSTMs?

- ✓ (a) LSTMs haben h (hidden state) UND c (cell state)
- (b) Nur einen Hidden State
- (c) Keine Gates
- (d) Löst Vanishing Gradient vollständig

3. VC-Dimension?

- ✓ (b) Maß für Komplexität/Kapazität des Modells
- (a) Anzahl Trainingsbeispiele
- (c) Validierungsgenauigkeit
- (d) Test-Loss

4. Autoencoder vs. PCA?

- ✓ (b) Autoencoder kann nichtlineare Repräsentationen lernen, PCA nur lineare
- (a) Umgekehrt
- (c) Äquivalent
- (d) PCA mehr Parameter

5. Sparse Coding?

- ✓ (a) Repräsentation als Vektor mit vielen Nullelementen
- (b) Dichte Vektoren
- (c) L2-Regularisierung
- (d) PCA-Verfahren

6. Approximation L0-Norm?

- ✓ (c) L1-Norm (konvex, fast überall differenzierbar)
- (a) L2-Norm
- (b) L_∞ -Norm
- (d) Frobenius-Norm

7. Transfer Learning?

- ✓ (a) Gewichte eines trainierten Netzes auf verwandte Aufgabe übertragen
- (b) Zwei Netze parallel
- (c) Dataaugmentation
- (d) Test-Anpassung

8. BPTT?

- ✓ (b) Lange Gradientenprodukte → Vanishing oder Exploding Gradients
- (a) Vorwärts propagiert
- (c) Ohne Kettenregel
- (d) Nur für lineare RNNs

2 Theorie — Stückweise lineare Funktion

(a) Vier lineare Gebiete

ReLU-Aktivierungen erzeugen 4 Gebiete basierend auf Vorzeichen von $(x_{\text{link}} + x_{\text{right}})$ und $(x_{\text{link}} - x_{\text{right}})$:

Gebiet 1: $(x_{\text{link}} + x_{\text{right}} > 0) \wedge (x_{\text{link}} - x_{\text{right}} > 0) \rightarrow y = (x_{\text{link}} + x_{\text{right}}) + (x_{\text{link}} - x_{\text{right}}) = 2x_{\text{link}}$

Gebiet 2: $(x_{\text{link}} + x_{\text{right}} > 0) \wedge (x_{\text{link}} - x_{\text{right}} \leq 0) \rightarrow y = (x_{\text{link}} + x_{\text{right}}) + 0 = x_{\text{link}} + x_{\text{right}}$

Gebiet 3: $(x_{\text{link}} + x_{\text{right}} \leq 0) \wedge (x_{\text{link}} - x_{\text{right}} > 0) \rightarrow y = 0 + (x_{\text{link}} - x_{\text{right}}) = x_{\text{link}} - x_{\text{right}}$

Gebiet 4: $(x_{\text{link}} + x_{\text{right}} \leq 0) \wedge (x_{\text{link}} - x_{\text{right}} \leq 0) \rightarrow y = 0 + 0 = 0$

Gebiet 1: $x_{\text{link}} > 0, |x_{\text{right}}| < x_{\text{link}}$ (rechts der y-Achse, zwischen den Diagonalen)

Gebiet 2: $x_{\text{link}} > 0, x_{\text{right}} > |x_{\text{link}}|$ (oberer Bereich, über beiden Diagonalen) $\rightarrow$ Nein: $x_{\text{link}} + x_{\text{right}} > 0 \wedge x_{\text{link}} < x_{\text{right}} \rightarrow$ oberes Dreieck

Gebiet 3: $x_{\text{link}} > x_{\text{right}} \wedge x_{\text{link}} + x_{\text{right}} < 0 \rightarrow$ unteres rechtes Dreieck

Gebiet 4: $x_{\text{link}} + x_{\text{right}} < 0 \wedge x_{\text{link}} < x_{\text{right}} \rightarrow$ linker Bereich

(b) Entscheidungsgrenze $y=1$ in jedem Gebiet

Gebiet 1: $2x_{\text{link}} = 1 \rightarrow x_{\text{link}} = 0.5$ (vertikale Linie)

Gebiet 2: $x_{\text{link}} + x_{\text{right}} = 1 \rightarrow$ Gerade mit Steigung -1

Gebiet 3: $x_{\text{link}} - x_{\text{right}} = 1 \rightarrow$ Gerade mit Steigung $+1$

Gebiet 4: $0 = 1$ — unmöglich, keine Grenze in diesem Gebiet

(c) Differenzierbarkeit

Nein, nicht überall differenzierbar. An den Grenzen der Gebiete (wo $x_{\text{link}} + x_{\text{right}} = 0$ oder $x_{\text{link}} - x_{\text{right}} = 0$) ist ReLU nicht differenzierbar (Knick). Innerhalb jedes Gebiets ist die Funktion linear und damit differenzierbar.

3 Theorie — CNNs

(a) Convolution vs. Cross-Correlation

Convolution: $(x \star w)(t) = \sum x(\tau) \cdot w(t-\tau)$ — Kernel wird gespiegelt.

Cross-Correlation: $(x \blacksquare w)(t) = \sum x(t+\tau) \cdot w(\tau)$ — keine Spiegelung.

PyTorch Conv-Layer implementieren Cross-Correlation. Da Kernel gelernt werden, ist Spiegelung irrelevant (das Netz kann gespiegelte Filter lernen).

(b) Residual Connections

$$x_{l+1} = x_l + \text{NN}_l(x_l; \theta_l)$$

Vorteil: Gradient kann direkt durch die Identitätsverbindung fließen (keine Multiplikation). Bei vielen Schichten: Gradienten verschwinden weniger. Ermöglicht Training sehr tiefer Netze (100+ Schichten).

(c) CNN Blueprint

$[\text{Conv} \rightarrow \text{Activation} \rightarrow \text{Pooling}] \times L \rightarrow \text{Global Average Pooling} \rightarrow \text{MLP}$

Global Average Pooling: Mittelt jede Feature Map auf einen Skalar $\rightarrow$ reduziert Overfitting, macht das Netz invariant gegenüber Eingabegröße.

MLP: Führt die eigentliche Klassifikation auf den extrahierten Features durch.

4 Theorie — Autoencoders

(a) Denoising Autoencoder

$$\min_{\{f, g\}} \frac{1}{N} \sum \|x_i - f(g(\text{noise}(x_i)))\|^2$$

Eine erste Schicht fügt Rauschen zum Input hinzu. Der Autoencoder muss den ursprünglichen Input rekonstruieren. Dadurch lernt er Invarianz gegenüber Rauschen und lokalen Korruptionen — robustere Repräsentationen.

(b) Anomalieerkennung mit Autoencoder

1. Trainiere Autoencoder nur auf normalen Daten. 2. Berechne Rekonstruktionsfehler $\|x - f(g(x))\|^2$ für neue Samples. 3. Hoher Fehler → Anomalie (Modell hat keine gute Repräsentation gelernt). 4. Rekonstruktionsfehler pro Pixel als Heatmap für Lokalisierung.

(c) Bottleneck-Tradeoff

Kleines Bottleneck: hohe Kompression, aber hoher Rekonstruktionsfehler (zu viel Information verloren).

Großes Bottleneck: niedrige Kompression, gute Rekonstruktion, aber schlechte Generalisierung (evtl. keine nützliche Repräsentation gelernt).

5 Programmierung — Adversarielle Beispiele

(a) Adversariales Beispiel erzeugen

```
def adversarial_example(x, t, f, n_steps=100, lr=0.01):
    z = x.clone().requires_grad_(True)
    optimizer = torch.optim.SGD([z], lr=lr)
    for _ in range(n_steps):
        optimizer.zero_grad()
        loss = (x - z).pow(2).sum() + torch.clamp(1 + t * f(z), min=0)
        loss.backward()
        optimizer.step()
    return z.detach()
```

(b) Batch-Anpassung

Minimale Änderung: x, t als Batch-Tensoren übergeben. Die Summen im Loss werden über den Batch gemittelt:

```
loss = (x - z).pow(2).sum(dim=1).mean() + torch.clamp(1 + t * f(z),
min=0).mean()
```

(c) Robustheit durch adversariale Beispiele

Adversarial Training: Erzeuge adversariale Beispiele während des Trainings und füge sie dem Trainingsdatensatz hinzu (Data Augmentation). Das Modell lernt so, auch auf adversarial perturbierten Inputs korrekt zu klassifizieren. → Erhöhte Robustheit, aber höhere Trainingskosten.